Milestone 3: Project Progress Report

Group RC 19: FPGA-Based Transformer Attention Accelerator

Satyarth | Rijul | Samarth | Om

Spring 2026

1. Introduction / Motivation

Transformers power most modern NLP systems, but running inference on an FPGA is hard. The attention mechanism combines dense matrix multiplies with nonlinear operations — softmax, layer normalization, activation functions — that a plain GEMM accelerator can't handle. We're building an INT8 accelerator on the Xilinx Alveo U55C that maps the full transformer decoder layer pipeline onto FPGA fabric: all linear layers through a shared MAC array, all nonlinear ops through dedicated hardware blocks.

After feedback from the course instructor — who pointed out that a standalone GEMM accelerator or offload chip architecture wasn't ambitious enough for a four-person grad project — we shifted focus from a pure matrix-multiply co-processor to the full attention pipeline. The MAC array we built remains the compute core, but the goal is now a complete decoder layer running end-to-end on the FPGA.

2. Overview / Application

Each transformer decoder layer runs a fixed sequence of operations. The linear ones all reduce to matrix multiplies, handled by our shared MAC array. The nonlinear ones need dedicated hardware:

1. Q/K/V Projections — X × Wq, X × Wk, X × Wv → Query, Key, Value matrices.
2. Attention Scores — $Q \times K^T$ scaled by $1/\sqrt{d_k}$, then softmax to get attention weights.
3. Weighted Sum — attention weights × V, followed by output projection × W_O.
4. FFN — two linear layers (W1 × x, W2 × act(h)) with a nonlinear activation between them.
5. Residual adds and layer normalization after each sub-block.

3. Architecture Diagram / Flow

The design targets an on-chip accelerator on the Alveo U55C: the full decoder layer runs on FPGA fabric with no PCIe round-trips mid-inference. A single MAC array time-multiplexes across every matrix multiply step; separate dedicated blocks handle the nonlinear operations.

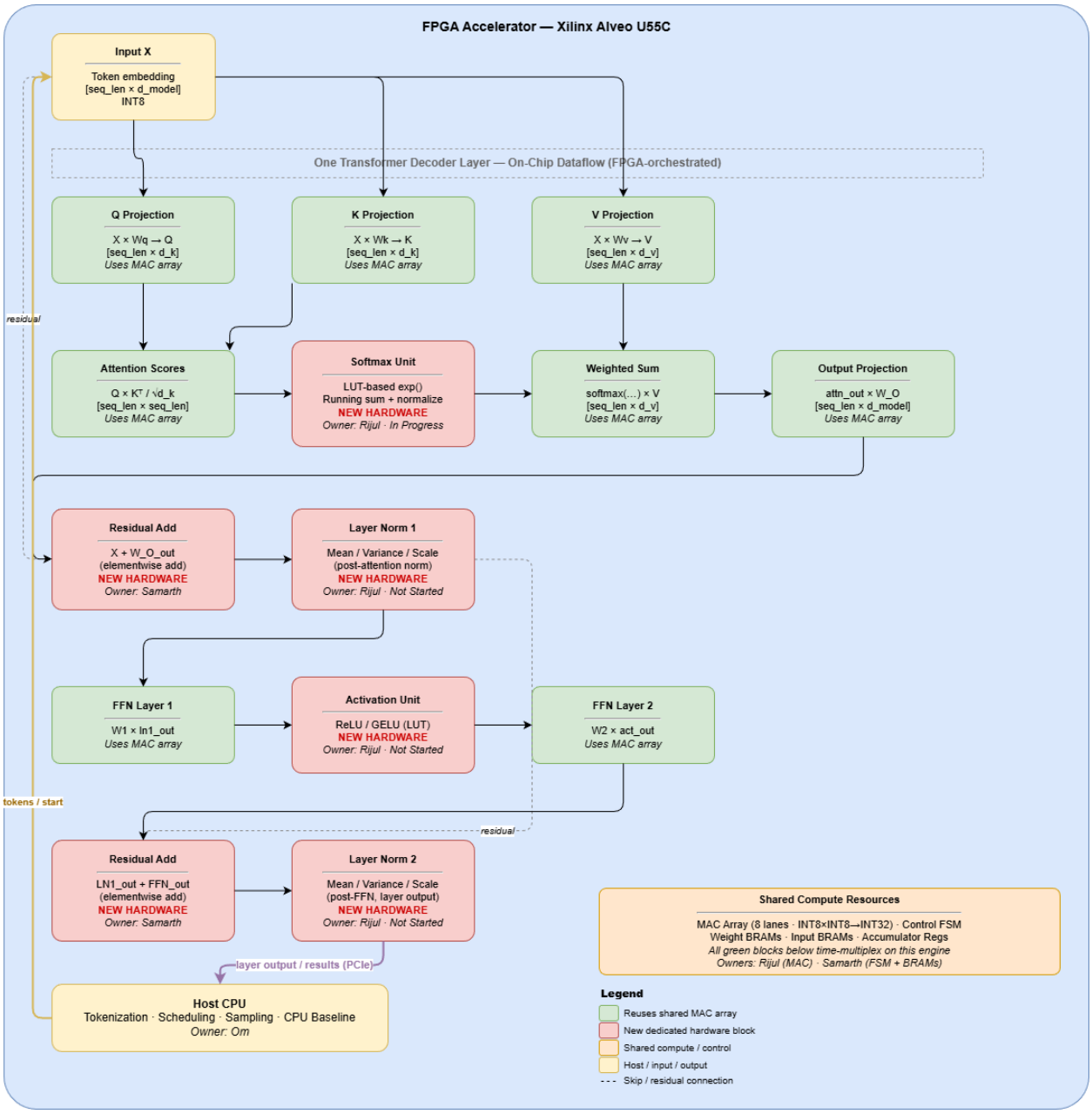

Figure 1: Full transformer decoder layer — QKV projections, attention, softmax, output projection, residual adds, layer norms, and FFN — all on-chip.

Shared Compute Engine: 8-lane INT8 × INT8 → INT32 MAC array. All matrix multiply steps (Q, K, V projections; $Q \times K^T/\sqrt{d_k}$; weighted sum × V; output projection × W_O; FFN1 and FFN2) time-multiplex through this engine. Output-stationary tiling: N/T tiles × K MAC cycles each.

Dedicated Hardware Blocks: Softmax unit (LUT-based exp, running sum, normalize); activation unit (LUT GELU / passthrough ReLU); two layer norm units (mean/variance/scale); two residual adders (elementwise add). These are combinational/pipelined units independent of the MAC array.

Control FSM: Five-state FSM (IDLE → LOAD → COMPUTE → WRITE → DONE) driving tiling, BRAM addressing, and accumulator control via k_idx and tile_idx counters (The FSM might need to be reworked).

Memory Subsystem: Active tile lives in on-chip BRAM: input vector (64 × 8b), weight banks × 8 (64 × 8b each), output buffer (64 × 32b). At scale, weights stream from HBM via double-buffered BRAM banks to overlap loading with compute.

Host Interface: PCIe host (XRT) sends input tokens and reads output logits; everything else stays on-chip. Currently uses behavioral write ports for testbench loading while Om integrates the XRT kernel.

Tools and Frameworks

Tool / Resource	Purpose
Vivado / Vitis 2024.1	Synthesis, implementation, and board integration
SystemVerilog	Primary HDL for all RTL modules
Icarus Verilog (iverilog)	Simulation and functional verification
Python / NumPy	INT8 model quantization, golden reference generation
C++ (x86 Host)	CPU baseline implementation for comparison
Xilinx Alveo U55C	Target FPGA platform (PCIe accelerator card)
GTKWave	VCD waveform viewing for debug

4. Tasks and Status

Task	Owner	Status	Notes
INT8 MAC unit design (mac_unit.sv)	Rijul	**Done**	Combinational, verified
8-lane parallel MAC array (mac_array.sv)	Rijul	**Done**	With accumulators and write-out
Tiling control FSM (control_fsm.sv)	Samarth	**Done**	5-state FSM, 7 checks pass
Top-level integration (top.sv)	Samarth	**Done**	FSM + MAC + BRAM stubs
FSM testbench (tb_control_fsm.sv)	Samarth	**Done**	7 automated checks, all pass
Integration testbench (tb_top.sv)	Samarth	**Done**	4 checks: all-ones, identity, done, re-run
MAC array testbench (tb_mac_array.sv)	Rijul	**Done**	Testbench written; Verified with test vectors
Block diagram and design documentation	Samarth	**Done**	FSM diagram, signal tables, cycle trace
Python quantized model + test vectors	Satyarth	**Done**	Done

U55C environment setup + PCIe host comm	Om	**In Progress**	XRT driver and host program
Softmax approximation hardware	Rijul	**In Progress**	LUT-based exp + piecewise linear
Attention dataflow scheduler on host	Samarth	**In Progress**	Orchestrate Q/K/V → attn → FFN sequence
Activation function unit (ReLU / GELU)	Rijul	**Not Started**	LUT for GELU, passthrough for ReLU
Layer normalization hardware	Rijul	**Not Started**	Mean/variance reduction + scaling
CPU baseline implementation (C++)	Om	**Not Started**	For speedup comparison
End-to-end attention block on FPGA	All	**Not Started**	Full Q/K/V + softmax + V multiply
Performance measurement and roofline analysis	All	**Not Started**	Latency, throughput, resource util.

5. Preliminary Results

All RTL modules pass simulation in Icarus Verilog.

FSM Verification (tb_control_fsm.sv)

tb_control_fsm.sv runs 7 automated checks for N=64, K=64, T=8. Key results:

- clear_acc pulses once per tile (8×), mac_valid HIGH for exactly 512 cycles (8 tiles × 64).
- done asserts at cycle 538 and de-asserts cleanly on the next cycle.

End-to-End Integration Verification (tb_top.sv)

tb_top.sv exercises the full datapath (FSM + MAC array + BRAM stubs) with four checks, all passing:

Check	Description	Expected	Result
CHECK_1	All-ones: x=[1,...,1], W=all 1s	y[i] = 64 for all i	**PASS**
CHECK_2	Identity: W=I, x[k]=k	y[i] = i for all i	**PASS**
CHECK_3	done de-asserts after 1 cycle	done=0 after DONE state	**PASS**
CHECK_4	Second run produces identical results	No stale accumulator state	**PASS**

The engine completes a 64×64 matrix-vector multiply in 538 cycles (512 MAC + 26 overhead).

6. Code / Tool Snapshots

All source code, testbenches, and documentation are maintained in a GitHub repository with clear per-module ownership and README documentation.